

Image Captioning com Modelos Multimodais: CLIP + Projetor + SmolLM2

Bruno Rodrigues

Licenciatura em Engenharia Informática e de Computadores

ISEL – Instituto Superior de Engenharia de Lisboa

Lisboa, Portugal

a52323@alunos.isel.pt

Resumo—Neste trabalho implementamos uma pipeline multimodal de *image captioning* que integra um encoder visual pré-treinado com um Large Language Model (LLM) através de um projetor treinável. Usamos o CLIP ViT-B/32 como encoder visual congelado, o SmolLM2-135M-Instruct como LLM congelado, e comparamos dois projetores: uma camada linear e um MLP de duas camadas com ativação GELU. O único módulo treinado é o projetor, com apenas 295K–1,1M parâmetros. A pipeline funciona exclusivamente em CPU, usando o dataset Flickr8k com um subconjunto de 2.000 imagens de treino. Os resultados mostram que o MLP converge mais rapidamente e atinge menor *loss* de validação (3,43 vs 4,50), com ROUGE-L de 0,10 vs 0,05 do projetor linear. O BLEU-4 é nulo em ambos os casos, refletindo as limitações de treinar apenas o projetor com dados reduzidos e um LLM com *priors* conversacionais. O trabalho demonstra a viabilidade da arquitetura e discute em detalhe as causas das limitações observadas.

Index Terms—image captioning, modelos multimodais, CLIP, projetor, language model, SmolLM2, Flickr8k

I. INTRODUÇÃO

A geração automática de legendas para imagens (*image captioning*) é uma das tarefas mais representativas na intersecção entre visão computacional e processamento de linguagem natural. O objetivo é produzir uma descrição textual coerente e relevante a partir do conteúdo visual de uma imagem — uma tarefa que exige compreensão simultânea de ambas as modalidades.

Nos últimos anos, a abordagem dominante evoluiu de modelos CNN+RNN [4] para arquiteturas que integram encoders visuais pré-treinados com Large Language Models (LLMs), como exemplificado pelo LLaVA [1] e pelo BLIP-2 [5]. Estas arquiteturas partilham uma ideia central: em vez de treinar um modelo do zero para ambas as modalidades, reutilizam encoders visuais e LLMs pré-treinados, ligando-os através de um módulo de alinhamento de dimensões — o *projetor* — que é o único componente treinável. Esta estratégia reduz drasticamente o número de parâmetros a otimizar e permite aproveitar representações já aprendidas em larga escala.

O principal desafio desta abordagem reside no *alinhamento de espaços de representação*: o encoder visual e o LLM foram treinados de forma independente, em dados e objetivos distintos, e os seus espaços de embedding não são diretamente compatíveis. O projetor tem de aprender a traduzir informação visual num formato que o LLM consiga interpretar como contexto para geração de texto.

Neste trabalho, implementamos uma pipeline multimodal leve seguindo este paradigma, com a restrição de funcionar exclusivamente em CPU. A modalidade escolhida é a imagem, e a tarefa é *image captioning* no dataset Flickr8k. A arquitetura consiste em:

- **Encoder visual:** CLIP ViT-B/32 [3], congelado, que produz embeddings visuais de 512 dimensões;
- **Projetor:** uma camada linear ou MLP que alinha o espaço visual (512 dims) com o espaço de embeddings do LLM (576 dims);
- **LLM:** SmolLM2-135M-Instruct, congelado, que recebe o embedding visual como prefixo e gera a legenda autoregressivamente.

O objetivo principal não é atingir o estado da arte, mas demonstrar que a arquitetura funciona e compreender o papel de cada componente — em particular, comparar a expressividade de um projetor linear com um MLP de duas camadas, e identificar as limitações desta abordagem quando aplicada com recursos computacionais reduzidos.

II. TRABALHOS RELACIONADOS

A. Image Captioning Clássico

Show and Tell [4] (Vinyals et al., 2015) foi um dos primeiros trabalhos a enquadrar *image captioning* como um problema de tradução sequência-a-sequência: uma CNN (GoogLeNet) extrai *features* visuais que inicializam um LSTM gerador de texto. O modelo é treinado de ponta a ponta com cross-entropy sobre as legendas. Esta abordagem estabeleceu o paradigma encoder-decoder e demonstrou que transferência de aprendizagem visual para geração de linguagem era viável, embora o módulo linguístico fosse treinado de raiz e não beneficiasse de LLMs pré-treinados.

B. Encoders Visuais Pré-treinados com Treino Contrastivo

CLIP [3] (Radford et al., 2021) introduziu o treino contrastivo em 400 milhões de pares imagem-texto recolhidos da internet (*WebImageText*), produzindo um encoder visual com representações ricas e transferíveis. O objetivo de treino é maximizar a similaridade coseno entre embeddings de imagens e legendas correspondentes, e minimizá-la para pares não correspondentes. O resultado é um espaço de representação conjunto onde imagens e textos semanticamente relacionados estão próximos. É o encoder que usamos neste trabalho: as

suas *features* são suficientemente informativas para *image captioning* sem qualquer fine-tuning adicional.

C. Integração de Visão com LLMs via Projetor

LLaVA [1] (Liu et al., 2023) é a referência mais direta para a nossa arquitetura. Usa CLIP ViT-L/14 como encoder visual e um projetor linear para mapear os embeddings visuais para o espaço de embeddings de um LLM (LLaMA/Vicuna). O treino é feito em duas fases: na primeira, treina apenas o projetor em pares imagem-texto de descrição simples; na segunda, faz *instruction tuning* de todo o modelo em dados de conversação multimodal. A nossa implementação é conceitualmente idêntica à primeira fase do LLaVA, com a diferença de que o LLM permanece congelado durante todo o processo.

BLIP-2 [5] (Li et al., 2023) propõe uma alternativa ao projetor simples: o Q-Former (*Querying Transformer*), um transformer ligeiro com um conjunto fixo de *query tokens* aprendíveis que interrogam as *features* visuais e produzem um número fixo de tokens visuais para o LLM. O Q-Former atua como gargalo de informação, comprimindo a representação visual e tornando-a mais compatível com o LLM. Esta abordagem é mais sofisticada que um projetor linear mas evita o custo de treinar o LLM completo.

LLaMA-VID [2] (Li et al., 2023) estende a ideia do projetor para vídeo, enfrentando o desafio de representar sequências de imagens de forma eficiente. A solução proposta representa cada *frame* com apenas 2 tokens no LLM — um token de contexto e um token de conteúdo —, permitindo processar vídeos longos sem explodir o comprimento do contexto. Mostra que a compressão extrema da informação visual é possível mesmo com projetores simples, desde que a informação seja adequadamente selecionada.

III. DATASET

A. Descrição

Usamos o dataset **Flickr8k**, disponível no HuggingFace (jxie/flickr8k). Contém 8.000 imagens de situações quotidianas — crianças a brincar, cães a correr, pessoas em atividades ao ar livre — cada uma anotada por 5 anotadores humanos independentes, totalizando 40.000 descrições em inglês. As legendas são tipicamente frases curtas de 10 a 15 palavras, descrevendo a ação principal e os sujeitos presentes na imagem. A diversidade de ângulos de descrição entre os 5 anotadores é aproveitada na avaliação: ao usar todas as referências, capturamos melhor a variabilidade natural da linguagem humana.

Tabela I
DIVISÕES DO DATASET FLICKR8K

Split	Total disponível	Usado (subset CPU)
Treino	6.000	2.000
Validação	1.000	400
Teste	1.000	400

B. Pré-processamento

O dataset é carregado diretamente do HuggingFace com a divisão predefinida. As 5 legendas por imagem estão disponíveis nas colunas `caption_0` a `caption_4`. Para treino, usamos apenas `caption_0` por imagem, seguindo a prática comum de usar uma única referência por exemplo de treino. Na avaliação BLEU e ROUGE, todas as 5 legendas são usadas como referências — prática padrão na literatura para Flickr8k, que permite avaliar cobertura face à variabilidade humana.

O pré-processamento das imagens é feito pelo `CLIPProcessor`: redimensionamento para 224×224 pixels e normalização com as médias e desvios padrão específicos do CLIP (média $[0.481, 0.457, 0.408]$, desvio $[0.269, 0.261, 0.276]$). As *features* CLIP são pré-calculadas uma única vez e guardadas em cache num ficheiro `.pkl` para eliminar o custo de inferência do encoder em cada epoch de treino.

O tokenizador do SmolLM2 é usado para converter as legendas em sequências de tokens de comprimento máximo 64, com *padding* à direita e truncagem quando necessário. As sequências incluem os tokens especiais `<bos>` e `<eos>` para delimitar o início e o fim da legenda.

IV. IMPLEMENTAÇÃO

A. Encoder Visual

Usamos o CLIP ViT-B/32 pré-treinado (openai/clip-vit-base-patch32), com 151,3M parâmetros totalmente congelados. A arquitetura ViT-B/32 divide a imagem em patches de 32×32 pixels, resultando em $(224/32)^2 = 49$ patches, cada um projetado para um vetor de 768 dimensões. O *transformer* visual processa estes patches com 12 camadas de *self-attention* e produz um vetor `[CLS]` que representa globalmente a imagem.

As *features* são extraídas através do caminho `vision_model → pooler_output → visual_projection`, onde a projeção visual do CLIP reduz as 768 dimensões internas para 512, alinhando o espaço de representação com o encoder de texto do CLIP. O resultado é um vetor $\mathbf{v} \in \mathbb{R}^{512}$ por imagem.

B. Projetor

O projetor é o único módulo treinável. Mapeia o espaço do CLIP ($d_{clip} = 512$) para o espaço de embeddings do SmolLM2 ($d_{llm} = 576$). A incompatibilidade de dimensões ($512 \neq 576$) impede a utilização direta das *features* CLIP como input do LLM, tornando o projetor necessário mesmo numa versão mínima. Testamos duas versões com complexidades diferentes:

Projetor Linear:

$$f_L(\mathbf{v}) = \mathbf{W}\mathbf{v} + \mathbf{b}, \quad \mathbf{W} \in \mathbb{R}^{576 \times 512} \quad (1)$$

com $576 \times 512 + 576 = 295.488$ parâmetros treináveis. É a solução mais simples possível — uma transformação linear que roda e escala o espaço visual.

Projektor MLP (2 camadas + GELU):

$$f_M(\mathbf{v}) = \mathbf{W}_2 \cdot \text{GELU}(\mathbf{W}_1 \mathbf{v} + \mathbf{b}_1) + \mathbf{b}_2 \quad (2)$$

com $\mathbf{W}_1 \in \mathbb{R}^{1024 \times 512}$ e $\mathbf{W}_2 \in \mathbb{R}^{576 \times 1024}$, totalizando 1.115.712 parâmetros. A camada oculta de 1024 unidades com ativação GELU introduz não-linearidade, permitindo ao projetor aprender transformações mais complexas entre os dois espaços. A ativação GELU (*Gaussian Error Linear Unit*) é preferida sobre ReLU em modelos de linguagem por produzir gradientes mais suaves.

C. Language Model

Usamos o SmolLM2-135M-Instruct (HuggingFaceTB/SmolLM2-135M-Instruct), com 134,5M parâmetros totalmente congelados. O modelo tem `hidden_size = 576`, 30 camadas transformer, 9 cabeças de atenção, e foi pré-treinado em dados textuais com fine-tuning de instrução. O modelo é carregado com `torch_dtype=torch.float32` para evitar conflitos de tipo com o projetor em ambientes CPU (que não suportam `bfloat16` de forma nativa).

A integração do sinal visual segue a abordagem do LLaVA: o embedding visual é concatenado como prefixo aos embeddings dos tokens da legenda no espaço de representação do LLM:

$$\mathbf{E}_{input} = [\mathbf{e}_{vis} \mid \mathbf{e}_{t_1}, \mathbf{e}_{t_2}, \dots, \mathbf{e}_{t_T}] \in \mathbb{R}^{(T+1) \times 576} \quad (3)$$

onde $\mathbf{e}_{vis} = f(\mathbf{v}) \in \mathbb{R}^{576}$ é o output do projetor e $\mathbf{e}_{t_i}$ são os embeddings dos tokens de texto obtidos pela camada de embedding do LLM. Esta sequência é passada diretamente ao LLM via `inputs_embeds`, contornando a camada de embedding de tokens.

D. Treino

O objetivo de treino é minimizar a *cross-entropy* causal calculada apenas nas posições dos tokens de texto. O token visual na posição 0 é mascarado com `label=-100`, indicando ao PyTorch para o ignorar no cálculo do loss. A máscara de atenção cobre a sequência completa (visual + texto), permitindo que o LLM atenda à informação visual ao gerar cada token:

$$\mathcal{L} = -\frac{1}{|T_{vis}|} \sum_{i=1}^T \mathbf{1}[l_i \neq -100] \log p(t_i \mid \mathbf{e}_{vis}, t_1, \dots, t_{i-1}) \quad (4)$$

Os hiperparâmetros são apresentados na Tabela II. A *learning rate* de 10^{-3} é deliberadamente elevada para forçar aprendizagem rápida do projetor em poucas épocas, dado o custo computacional em CPU.

E. Inferência

Na fase de inferência, a geração de legendas é feita de forma autoregressiva com *greedy decoding*. O processo começa com o embedding visual como único contexto e gera tokens um a um até atingir o token `<eos>` ou o comprimento máximo de 64 tokens:

Tabela II
HIPERPARÂMETROS DE TREINO

Parâmetro	Valor
Épocas	5
Batch size	8
Learning rate	1×10^{-3}
Optimizer	AdamW
Max tokens	64
dtype	float32
Caption de treino	caption_0
Subset de treino	2.000 imagens

- 1) Calcular $\mathbf{e}_{vis} = f(\mathbf{v})$ e inicializar a sequência com $[\mathbf{e}_{vis}]$;
- 2) Para cada passo i : passar a sequência atual ao LLM, obter logits na última posição, selecionar o token com maior probabilidade (`arg max`);
- 3) Converter o token em embedding e concatenar à sequência;
- 4) Parar quando o token gerado for `<eos>` ou `<pad>`.

A ausência de *KV-cache* torna a inferência $O(T^2)$ em tempo mas simplifica a implementação. O *greedy decoding* é determinístico e adequado para avaliação comparativa entre projetores.

V. RESULTADOS

A. Curvas de Loss

A Tabela III apresenta as curvas de loss de treino e validação para ambos os projetores ao longo das 5 épocas. O MLP converge mais rapidamente logo na primeira época (4,40 vs 5,25 de *train loss*) e continua a melhorar consistentemente, atingindo 3,40 no final do treino contra 4,55 do projetor linear. Em ambos os casos, a *val loss* acompanha a *train loss* sem sinais de *overfitting* significativo, o que é esperado dado o número reduzido de parâmetros treináveis.

Tabela III
LOSS DE TREINO E VALIDAÇÃO POR ÉPOCA

Época	Projektor Linear		Projektor MLP	
	Treino	Val	Treino	Val
1	5,251	5,184	4,404	4,117
2	5,209	5,282	3,879	3,652
3	5,137	4,879	3,557	3,536
4	4,740	4,634	3,473	3,468
5	4,553	4,500	3,404	3,434

B. Métricas de Avaliação

Usamos duas métricas standard em *image captioning*:

BLEU-4 (*Bilingual Evaluation Understudy*) mede a precisão de 4-gramas da legenda gerada face às referências humanas. Um valor de 0 indica que nenhuma subsequência de 4 palavras consecutivas gerada pelo modelo coincide com qualquer referência. É uma métrica severa e mais adequada para modelos que já produzem texto fluente.

ROUGE-L (*Recall-Oriented Understudy for Gisting Evaluation*) mede a maior subsequência comum (*LCS*) entre a legenda gerada e as referências. É mais tolerante a reordenações e menos sensível a fluência do que o BLEU, sendo mais informativa quando o modelo ainda produz texto imperfeito.

Tabela IV
RESULTADOS NO CONJUNTO DE TESTE (400 IMAGENS, 5 REFERÊNCIAS POR IMAGEM)

Projektor	BLEU-4	ROUGE-L
Linear	0,0000	0,0521
MLP	0,0000	0,1023

O BLEU-4 é 0 em ambos os casos, indicando que as legendas geradas não partilham sequências de 4-gramas com as referências humanas. O ROUGE-L mostra que o MLP é aproximadamente $2\times$ melhor que o linear (0,10 vs 0,05), confirmando que a maior expressividade do MLP se traduz em melhores outputs mesmo com métricas de texto.

C. Análise Qualitativa

A análise das legendas geradas revela dois modos de falha sistemáticos:

Degeneração em repetição: o modelo colapsa para sequências repetitivas, como “1 2 3 4 5 6 7 8 9 10 ...” ou repetição de tokens de pontuação. Este comportamento é conhecido na literatura como *repetition loop* e ocorre quando o modelo não consegue escapar de atratores locais no espaço de geração.

Texto conversacional: o LLM, que foi fine-tunado como assistente de instrução, ativa os seus *priors* conversacionais em vez de gerar legendas descritivas. Exemplos observados incluem “I’m so glad you’re doing well! How can I help you today?” ou “Sure! Here’s a summary of what you asked for.” — respostas que fazem sentido num contexto de chat mas não como descrições de imagens.

Nenhum dos projetores gerou legendas que descrevessem corretamente o conteúdo visual das imagens do conjunto de teste. O MLP apresenta outputs ligeiramente mais variados, o que explica o ROUGE-L superior, mas sem qualidade semântica.

D. Discussão

Os resultados refletem três limitações fundamentais desta implementação:

1. Dataset de treino insuficiente. Com apenas 2.000 exemplos de treino, o projetor vê muito poucas amostras para aprender um mapeamento robusto entre o espaço visual e o espaço linguístico. O LLaVA [1] usa 595.000 exemplos na fase de alinhamento. A diferença é de $\approx 300\times$.

2. LLM congelado com *priors* conversacionais. O SmolLM2-Instruct foi fine-tunado para responder a instruções em formato de chat. Com o LLM congelado, o projetor tem de competir com estes *priors* muito fortes. Sem *instruction tuning* adicional no formato de caption, o LLM interpreta o embedding visual como um prompt de chat e responde em conformidade.

3. Representação visual com um único token. Toda a informação visual está comprimida num único vetor de 576 dimensões. Modelos estado-da-arte usam dezenas a centenas de tokens visuais para preservar informação espacial. Um único token de contexto é insuficiente para descrever cenas complexas.

A diferença de *loss* entre os dois projetores (3,40 vs 4,55) indica que o MLP aprende um mapeamento superior. Com mais dados e *instruction tuning*, esta diferença provavelmente traduziria em ganhos significativos nas métricas.

VI. CONCLUSÃO

Implementámos uma pipeline multimodal de *image captioning* funcional, com apenas 295K–1,1M parâmetros treináveis, usando CLIP ViT-B/32 como encoder visual congelado e SmolLM2-135M-Instruct como LLM congelado, ligados por um projetor linear ou MLP. O treino corre exclusivamente em CPU com um subconjunto do Flickr8k.

Os resultados quantitativos são fracos (BLEU-4=0, ROUGE-L $\leq$ 0,10), mas as curvas de *loss* demonstram que o sistema aprende: a *loss* decresce de forma consistente em ambos os projetores, e o MLP converge para valores substancialmente mais baixos. A fraca qualidade das legendas geradas reflete limitações de recursos — dados insuficientes, LLM congelado com *priors* conversacionais, e representação visual com um único token — e não uma falha de arquitetura. A abordagem é conceitualmente idêntica à fase 1 do LLaVA [1], que com ordens de grandeza mais dados e parâmetros treináveis atinge o estado da arte.

O principal contributo deste trabalho é a demonstração de que a arquitectura encoder–projetor–LLM é implementável com recursos mínimos, e que o MLP supera o projetor linear mesmo em condições adversas.

VII. TRABALHO FUTURO

Os resultados obtidos apontam claramente para várias direções de melhoria:

Fine-tuning parcial do LLM com LoRA. O problema mais crítico identificado é o conflito entre os *priors* conversacionais do SmolLM2-Instruct e o formato de caption esperado. Uma solução promissora é o *Low-Rank Adaptation* (LoRA), que permite fazer fine-tuning de uma pequena fração dos parâmetros do LLM (tipicamente $< 1\%$) com custo computacional reduzido. Isto permitiria ao modelo aprender o formato de legenda sem perder as capacidades linguísticas gerais.

Treino com dataset completo e GPU. O subconjunto de 2.000 imagens é claramente insuficiente. Treinar com as 6.000 imagens completas do Flickr8k, ou idealmente com o Flickr30k (31.783 imagens), num ambiente com GPU, permitiria avaliar se a arquitetura tem potencial real. A diferença de velocidade entre CPU e GPU em operações de matriz é tipicamente 10–50 $\times$, tornando o treino com mais épocas e mais dados viável.

Múltiplos tokens visuais. Em vez de comprimir toda a informação visual num único token, seria possível usar os 49 tokens de patch do ViT diretamente (antes do

pooler_output), projetando cada patch para o espaço do LLM. Isto preservaria informação espacial e permitiria ao LLM “olhar” para diferentes regiões da imagem ao gerar cada palavra.

Beam search na inferência. O *greedy decoding* tende a colapsar para sequências de alta frequência. O *beam search* com $k = 5$ beams exploraria múltiplas hipóteses em paralelo e tenderia a produzir legendas mais fluentes e diversas.

Uso do BLIP-2 como projetor. Substituir o projetor linear/MLP por um Q-Former pré-treinado (como proposto em BLIP-2 [5]) introduziria um módulo de alinhamento já treinado em larga escala, potencialmente reduzindo a quantidade de dados necessária para fine-tuning.

REFERÊNCIAS

- [1] H. Liu, C. Li, Q. Wu, and Y. J. Lee, “Visual Instruction Tuning,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. [Online]. Available: <https://arxiv.org/abs/2304.08485>
- [2] K. Li, Y. He, Y. Wang, Y. Li, W. Wang, P. Luo, Y. Wang, L. Wang, and Y. Qiao, “LLaMA-VID: An Image is Worth 2 Tokens in Large Language Models,” in *Proc. European Conf. Computer Vision (ECCV)*, 2024. [Online]. Available: <https://arxiv.org/abs/2311.17043>
- [3] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever, “Learning Transferable Visual Models From Natural Language Supervision,” in *Proc. Int. Conf. Machine Learning (ICML)*, 2021. [Online]. Available: <https://arxiv.org/abs/2103.00020>
- [4] O. Vinyals, A. Toshev, S. Bengio, and D. Erhan, “Show and Tell: A Neural Image Caption Generator,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2015. [Online]. Available: <https://arxiv.org/abs/1411.4555>
- [5] J. Li, D. Li, S. Savarese, and S. Hoi, “BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models,” in *Proc. Int. Conf. Machine Learning (ICML)*, 2023. [Online]. Available: <https://arxiv.org/abs/2301.12597>